

Towards Transparent, Reusable, and Customizable Data Science in Computational Notebooks

Frederick Choi*

fc20@illinois.edu

University of Illinois at Urbana-Champaign
Urbana, Illinois, USA

Hannah Kim

hannah@megagon.ai

Megagon Labs

Mountain View, California, USA

Sajjadur Rahman

sajjadur@megagon.ai

Megagon Labs

Mountain View, California, USA

Dan Zhang

dan_z@megagon.ai

Megagon Labs

Mountain View, California, USA

ABSTRACT

Data science workflows are human-centered processes involving on-demand programming and analysis. While programmable and interactive interfaces such as widgets embedded within computational notebooks are suitable for these workflows, they lack robust state management capabilities and do not support user-defined customization of the interactive components. The absence of such capabilities hinders workflow reusability and transparency while limiting the scope of exploration of the end-users. In response, we developed MAGNETON, a framework for authoring interactive widgets within computational notebooks that enables transparent, reusable, and customizable data science workflows. The framework enhances existing widgets to support fine-grained interaction history management, reusable states, and user-defined customizations. We conducted three case studies in a real-world knowledge graph construction and serving platform to evaluate the effectiveness of these widgets. Based on the observations, we discuss future implications of employing MAGNETON widgets for general-purpose data science workflows.

CCS CONCEPTS

• **Human-centered computing** → **Interactive systems and tools**; **Visualization**; • **Information systems**;

KEYWORDS

Literate Programming; Exploratory Programming; Interactive programming; Data Science; Computational notebooks;

ACM Reference Format:

Frederick Choi, Sajjadur Rahman, Hannah Kim, and Dan Zhang. 2023. Towards Transparent, Reusable, and Customizable Data Science in Computational Notebooks. In *Extended Abstracts of the 2023 CHI Conference on*

Human Factors in Computing Systems (CHI EA '23), April 23–28, 2023, Hamburg, Germany. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3544549.3585807>

1 INTRODUCTION

Data science workflows are iterative wherein users often switch between multiple tools, including programming environments (e.g., Jupyter Notebooks), visualization tools (e.g., Tableau or PowerBI), and spreadsheets (e.g., Excel) [32, 41]. Such back-and-forth switching results in a discontinuous workflow — users are forced to execute repetitive *glue* tasks manually to bridge the gap between two systems during each switch [8]. The overhead of frequent context switching discourages users from using analysis tools and restricts them to working only in code [41]. Alspaugh et al. [2] advocated for systems that combine the expressivity of coding platforms and the ease of use of visual analysis tools. Computational notebooks (e.g., Jupyter [23] and Observable [15]) embedded with interactive interfaces called *widgets* [17] support these objectives. Data practitioners employ these interfaces for auditing, exploring, and sharing data insights [5, 22, 32, 42].

However, gaps remain with respect to transparency, reusability, and customizability of user actions while using these programmable and interactive interfaces. Widgets operate as embeddable and lightweight interfaces with interactive components. User actions on the front-end trigger pre-defined operations called *data operations* that update the widget state and re-render components accordingly. The widget state maintains the values of the front-end component properties, e.g., frequency distribution corresponding to a bar chart. Figure 1B displays such an interface that summarizes a graph database — the component on the left displays the schema with node and relation types, and the two components on the right shows the corresponding node (top) and relation (bottom) distributions. Clicking a node type in the schema triggers data operations that recompute the corresponding distributions of the bar charts. Existing widgets such as *ipywidgets* [17] only maintain the most recent state and do not instrument mechanisms to track the state transitions triggered by user interactions. Therefore, these widgets lack transparency as the history of interactions and their corresponding states are lost and, reusability as recovering previous states requires users to execute the interactions from scratch. Lack of such state management capabilities contribute to loss of knowledge when data science teams share information [44] and limit the reproducibility

*Work done during internship at Megagon Labs.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

CHI EA '23, April 23–28, 2023, Hamburg, Germany

© 2023 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-9422-2/23/04.

<https://doi.org/10.1145/3544549.3585807>

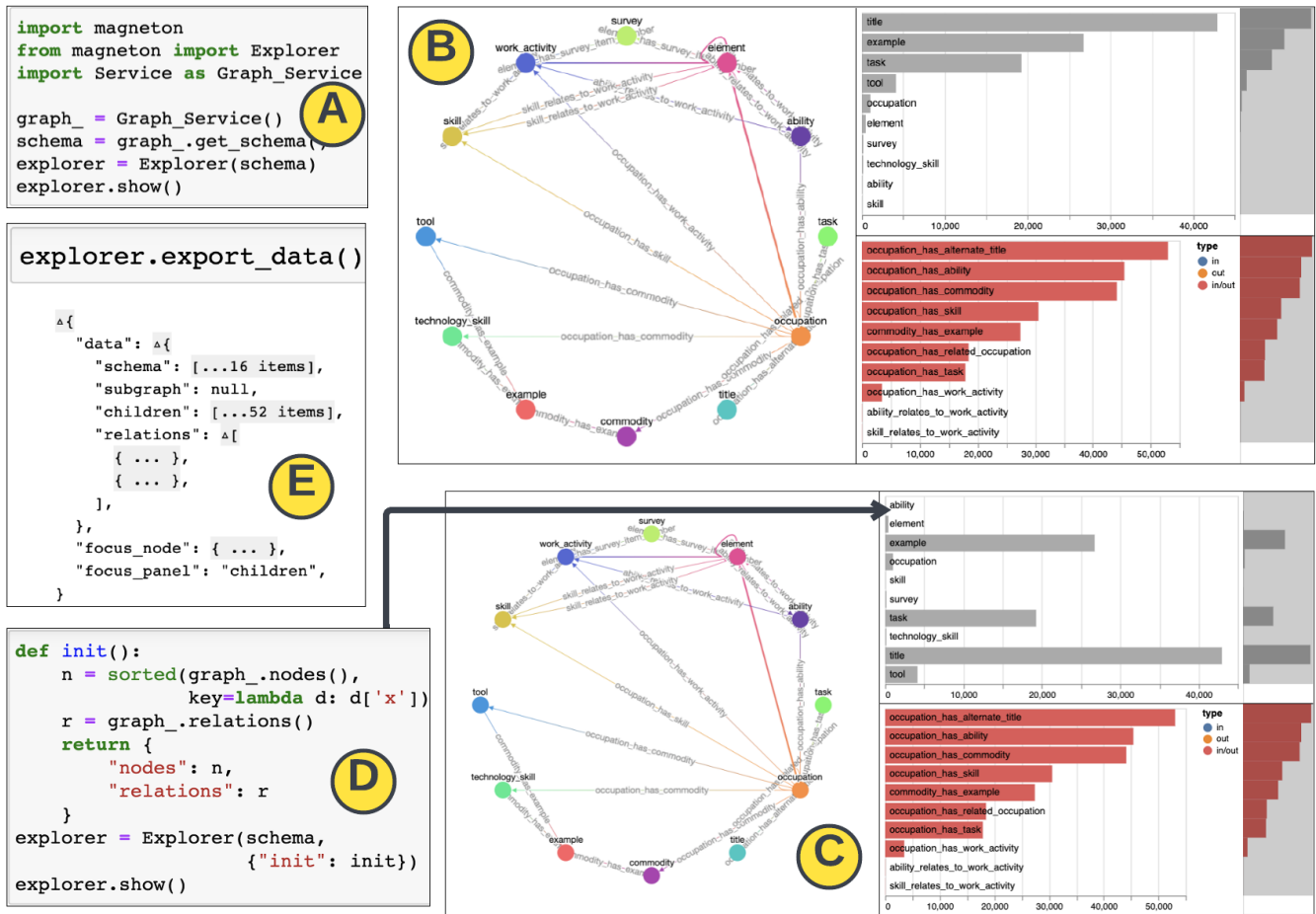


Figure 1: Overview of MAGNETON features (counter clock-wise). (A) User instantiates a graph exploration widget from the notebook. **(B)** A multiple-coordinated view consisting of a graph schema and corresponding node (top) and relation (bottom) distribution components is displayed. **(C)** A customized widget displaying node distribution in alphabetic order – **(D)** user defines an initialization function `init()` to customize the sort order and passes it as a callback function during widget initialization. **(E)** User exports the widget state using the `export_data()` accessor function.

of workflows [19, 20, 33]. Furthermore, these interfaces lack the affordances for end-users to customize the built-in data operations. For example, users cannot override the data operation to update the sort order of the bar charts in Figure 1B from descending order of frequency to alphabetic order of the labels of the bars. However, existing work advocates for such on-demand customizations to ensure more flexibility in exploring diverse objectives [33, 46].

To address these gaps, we implement MAGNETON¹, a framework for authoring programmable and interactive interfaces equipped with built-in state management and on-demand customization capabilities. These interfaces are developed by extending existing widgets. Users can author their workflows in the Jupyter Notebook (Figure 1A) by instantiating task-specific widgets (Figure 1B). Users can **customize** an interaction’s underlying data operation – *e.g.*,

the top bar chart in Figure 1C — by writing custom code in the notebook (Figure 1D). Each widget has a built-in history view, enabling users to explore their interaction history and access corresponding states, thereby ensuring **transparency** and **reusability** of widget states. Users can export the widget state programmatically as JSON objects (Figure 1E).

We implemented a suite of MAGNETON widgets to support various tasks within an in-house knowledge graph construction and serving platform at Megagon Labs, such as graph curation and knowledge integration. We conducted three preliminary case studies involving these tasks where data practitioners at Megagon Labs used MAGNETON within their real-world workflows. The interactive interfaces embedded within computational notebooks positively impacted their experiences and helped uncover interesting insights which remained unnoticed in their regular workflows without MAGNETON. Examples include identifying low-quality knowledge acquisition candidates and incorrect knowledge integration

¹Similar to Magnetron, a robot-like Pokémon, our proposed framework stitches together three objectives (Magnetite): transparency, reusability, and customization, to enable robust *programmable and interactive* interfaces in computational notebooks.

recommendations. Participants found the on-demand customization feature empowering and the ability to access interactions states helpful in managing *glue* tasks between various steps within their workflows. However, participants also identified several limitations that encourage fundamental research questions related to the learning curve, composing widgets, and balancing automation with user agency. We open-source the MAGNETON framework at <https://github.com/megagonlabs/magneton>.

2 RELATED WORK

Data practitioners often use interactive programming environments which enable interactive exploration of data [21, 33]. Computational notebooks are examples of such environments commonly used by practitioners. Charting libraries such as Altair [40] and Plotly [30] also enable interactive visualization. However, unlike MAGNETON these libraries lack affordances for customizing the data operations and do not capture interaction history. B2 [42], a dataframe wrapper with chart recommendation capabilities, captures interaction history by transforming UI interactions into dataframe operations and appending those operations in a notebook cell. The intermediate states can be reproduced by uncommenting and executing a python statement which is a cumbersome experience. Moreover, removing the notebook cell erases the interaction history. Therefore, additional version control mechanisms are required to achieve true persistence [7]. Additionally, the mapping from user interactions to dataframe operations are pre-defined and cannot be customized. In contrast, MAGNETON extends widgets in computational notebooks to enable persistent interaction history and on-demand customization. While B2 is limited to tabular dataframes, MAGNETON may support other data domains as users can plug in any functions to overwrite the underlying data operation corresponding to an interaction. *mage* [22] is another tool similar to B2 that translates interactions on interactive components within widgets to code, e.g., dataframe operations, and exhibits similar limitations. Other interactive programming environments are not limited to the notebook paradigm. For example, interactions with visualizations in GUESS [1] and Leam [12, 32] are captured in a Python environment. While visualizations in these tools are programmable via declarative commands, they do not track and persist the user's interaction history, thereby impeding transparency and reusability of user actions. Variolite [18] enables users to explore their previous interactions with code only. However, these bespoke tools require users to transition to and learn a new platform.

Frameworks such as Panel [13], Plotly Dash [29], and Symphony [5] use independent components to create visualizations that can be used in both Jupyter notebooks and standalone websites. However, Plotly Dash does not easily extend to custom visualizations, unlike Panel and Symphony. MAGNETON also supports a wider range of visualization libraries. However, these frameworks lack support for on-demand customization of underlying data operations and do not track interaction history, both of which are supported as built-in features by MAGNETON widgets. Streamlit [16] is another platform for generating interactive web dashboards using Python script. However, the platform prioritizes web applications rather than exploratory data science and does not focus on objectives such

as transparency, reusability, and customization. Moreover, a significant learning curve is associated with learning a new platform.

3 MAGNETON FRAMEWORK DESIGN

We now explain how MAGNETON helps author widgets that support transparent, reusable, and customizable user actions.

3.1 Widget Frameworks: Design and Limitations

Widgets are interactive elements, e.g., sliders, text boxes, buttons, that have representations both in the kernel, i.e., where code is executed, and the front-end, i.e., the notebook web interface. However, recent frameworks for authoring widgets [14] also enable integration of interactive dashboards in the front-end [5, 42, 45].

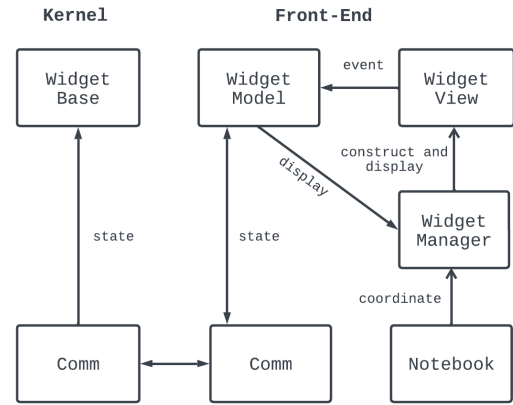


Figure 2: Design of basic, i.e., traditional widgets.

As shown in Figure 2, Widgets (e.g., *ipywidgets* [17]) maintain their state both at the back-end kernel (called *Widget Base*) and the front-end (called *Widget Model*.) The *Widget Base* and *Widget Model* remain in-sync via the communication API called *Comm*. However, only the most recent state is maintained, making the widgets essentially *memoryless*. The *Widget Manager* coordinates the display of the widget in the front-end *Widget View*. The *Widget View* is a container for rendering interactive components using front-end libraries and web frameworks. The *Widget View* only registers low-level event listeners corresponding to user interactions on the components. For example, a *selection* interaction on the graph node in Figure 1B that updates the bar charts is registered as an *onClick* listener. Therefore, these widgets are *agnostic* of the user's high-level interaction types and additional context, such as where the interaction happened and which components were updated. The *memoryless* and *interaction agnostic* nature of widgets prevent tracking of the user's interaction history and the corresponding widget states. Moreover, such a design primarily serves to parameterize data operations in the kernel using front-end events — a widget state variable (e.g., current node identifier) impacted by a low-level event (e.g., *onClick*) serves as an input parameter to a data operation (e.g., distribution computation). Any change in the widget variable triggers a recomputation of the data operation. In the notebook, the users can programmatically access and update the parameters of the data operations. However, the

framework, developers can define a data operation to be shared. For example, a shared data operation may return a distribution sorted by descending order of frequency. However, the user may prefer viewing the distribution in the alphabetic order of labels. As shown in Figure 1C and 1D, a user-defined function (UDF) written in the notebook — which reflects the updated sort order — is mapped to these the actions during widget instantiation time. In the kernel, the state manager parses the UDFs using custom serializers and overrides the data operation corresponding to the shared action. Such a design expands the “events parameterizing code” paradigm of widgets to “operations parameterizing code” and offers more flexible customization capabilities — users can keep updating the shared actions to explore different objectives by modifying the function defined in the notebook. Note that developers may implement data operations such as schema generation and distributions computation using standalone libraries or from scratch. In the case studies described in Section 4, we used an in-house graph query library, which was published as a Python package.

Components in MAGNETON Widget View. We use the React web framework [36] to develop the front-end components and the IDOM-Jupyter package [14] for component rendering in the Widget View. The components are TypeScript [39] modules that enable the rendering of a wide range web-based visualization libraries. For example, we used a custom graph visualization library to render the schema graph [10], Vega-lite [38] to render the bar charts, and a JavaScript library to render tables. As TypeScript supports static typing, developers can define application-specific data types and use those across the modules. Therefore, using TypeScript ensures a tighter integration between the Widget Base in the kernel and the Widget Model in the front-end. Moreover, when customizing data operations defined as shared actions, the pre-defined types provide hints to the user about the expected return type of the customized function. Each of the components rendered in the Widget View is fully interactive. These interactions, derived from existing visualization research [3, 43] are reactively synchronized across MAGNETON components, enabling multiple-coordinated visualizations (e.g., Figure 1B.)



Figure 3: Design of MAGNETON widgets. The dashed (“-”) elements, *i.e.*, the stateful widget and widget view wrappers, are introduced by MAGNETON.

Since data operations in the kernel correspond to user interactions in the front-end component, we introduce the concept of *shared actions*. Shared actions are data operations that end-users can override from the notebook. The operation definitions are essentially shared between the kernel and front-end. In the MAGNETON



Figure 4: The history view of a widget (`widget.history.show()`). Clicking the *Restore* button loads previous state visualizations.

Case Study Details. Two case studies involved graph exploration for curating knowledge graphs and identifying opportunities for graph expansion. For graph exploration, the participants employed an *Explorer* widget (Figure 1A). To explore the graph, the participants performed (*P1* and *P2*) various interactions on the widget components. Examples include panning and zooming to understand the graph schema and selection to view node and relation distribution which provides additional details helpful for uncovering inconsistencies that require further curation. In the final case study, the participant (*P3*) performed a knowledge integration task where they focused on verifying alignment candidates extracted from a text corpus and assigning merging decisions, such as insert, ignore, and defer, by exploring candidate entities in the in-house knowledge graph. We provided an *alignment-verification* widget for the verification task (see Figure 5). The widget contains an interactive table component with alignment candidates from the text corpus and the graph in two columns and a selection menu of alignment decisions in another column. The widget also displays the context of alignment candidates — a table component showing descriptions of an entity extracted from the text and a graph component displaying the sub-graph of the corresponding graph node.

4.1 Study Observations

MAGNETON Widget Usage. The interactive widget-based setup enabled the participant to stitch together different tasks in the same ecosystem, such as writing code and interactively exploring data. In their previous setting, all participants had to switch among multiple tools, such as spreadsheets, notebooks or IDEs, and Neo4j graph browser [27], which was cumbersome. *P3* commented — “... writing code and visualization; this interactive and graphical feature is much better.” *P3* appreciated the ability to view the alignment candidates in the context of the corpus and graph (as shown by the bottom two components in Figure 5) and make decisions more confidently. Participants (*P1* and *P2*) were able to identify low-quality long-tail nodes and relations in the graph using the exploration widget. The multiple-coordinated views helped explore node and relation distributions — low-frequency nodes in the graph were good candidates for further expansion. *P1* commented — “I like the feature that you can filter the graph by selecting a node and a corresponding incoming or outgoing relation.”

Impact of MAGNETON Features. One participant (*P1*) greatly appreciated the general features of MAGNETON widgets, such as the history view, which enabled them to explore their interaction history to revisit specific states in the widget. *P1* mentioned — “I wish that it (history view) were integrated within Neo4j browser”. Moreover, the data accessor feature helped participants access the data underlying a given state. *P1* also commented that the insights obtained through the history view and data obtained through the accessor functions could be shared with other team members during team meetings. *P2* utilized the shared action feature to override the data operation underlying the distribution component. They employed a sort wrapper to customize the order of the bars alphabetically to access nodes that the participant was interested in quickly. For example, the participant was interested in a node whose title started with “c” and could not scroll and locate the node among approximately 1000 bars in the distribution component. The feature helped improve the discoverability of desired information in the presence of too many bars. *P3* also used the shared action feature to declutter the “sub-graph” component visualization within the verification widget: instead of showing the entire node neighborhood, which may seem cluttered, the participant used a filtering function — as a wrapper of the underlying sub-graph computation data operation — to only display the node, and its parent. The participant characterized the experience as *transient customizability*: “When I am exploring, I am not attached to one objective. Right now, I am looking at only the parent (and the node), but later maybe I want to also view siblings ... so it can be ephemeral. I like the transient customizability.”

Limitations. The participants reported several usability issues with the MAGNETON widgets. *P3* suggested adding interactions such as filtering and grouping to enable more effective exploration of the interaction states in the history view. *P2* requested mechanisms to avoid scrolling through the entire interaction history, “it would be helpful to add a bookmark feature similar to web browsers.” While the participants utilized the shared action feature to further customize data operations in widgets, they requested adding more features out-of-the-box to minimize such customization requirements wherever possible. *P2* commented — “the shared action feature is helpful, but a sort could have been avoided had there been a search feature in the bar chart component.” Moreover, to help design UDFs for overriding shared actions, the participants (*P2* and

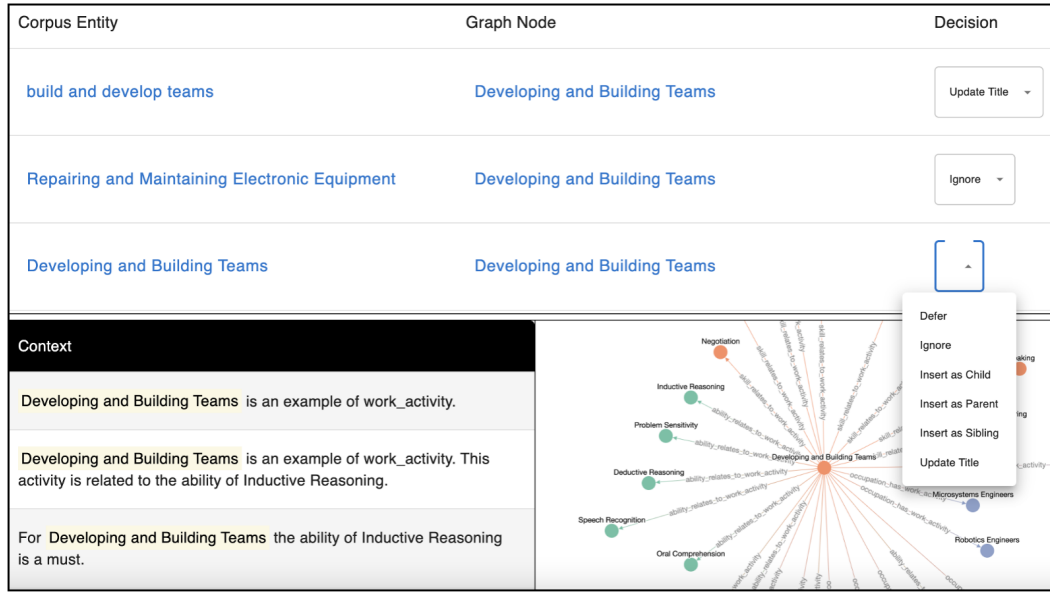


Figure 5: Alignment verification widget.

P3) also requested more clarification in the form of “standardized documentation” to explain the components of the widget, their data types, and the supported interactions.

5 DISCUSSION

Study limitations. We acknowledge that the scope of our evaluation is limited, specifically targeting the graph domain in the broader landscape of data science workflows while focusing on an industrial setting. Moreover, the study participants were employed at a single company. While the observed practices exist widely in industry and academia, the choice of participants inevitably impacted the generalizability of the findings due to organizational norms, policies, and infrastructures. Additional studies could explore MAGNETON’s usage benefits and limitations in diverse settings.

Documentation and usage. Even so, the observed usage benefits and limitations of case studies highlight several opportunities for improvement. While participants appreciated the freedom of customizing operations that helped accommodate ephemeral and ever-evolving objectives within their exploratory analysis, there were suggestions for adding more features by default. This tension indicates the inherent trade-off between user empowerment and user-friendliness. We are conducting participatory design studies within Megagon Labs to identify features that could be added as built-in functionalities of the widgets. The study also aims to identify additional components for supporting new workflows within the graph construction and serving platform. To improve the comprehension of a widget’s features, we plan to implement a *describe* method to explain component definitions, *i.e.*, their data types, supported interactions and their corresponding data operations, and the scope of the shared actions.

Authoring strategies for widget developers. One of the key strengths of the MAGNETON framework is the capability to compose task-oriented widgets by combining components. Therefore, a possible research direction can be to investigate various component authoring strategies for widget developers — for example, via declarative specification (*e.g.*, by leveraging the grammar of interactive graphics [38]) or using direct manipulation-based (*e.g.*, Lyra [37]). Graphileon [11] is a graph-driven dashboard development environment that uses a graph database to store user-interface components (*e.g.*, Networks, Tables, Forms) in nodes. Events modeled as relations define the interactions and data flows between the UI components. Future research may explore such view composition strategies for MAGNETON widgets.

Utilizing interaction provenance. As highlighted in the case studies, interaction on the interface, even with specific tasks and goals, may lead to many events being recorded. As a result, the history view may become challenging to use due to perceptual scalability limitations [6, 26, 31]. Future versions of MAGNETON can enhance the history view to support typical exploratory data analysis operations such as search and filter. Moreover, these widgets capture the context (where and when an interaction occurred) and scope (specific data domain, *e.g.*, graph) of interactions. Managing such interaction provenance may have additional benefits for intelligent agents that utilize interaction history. For instance, Solas [9] may leverage rich interaction history across sessions to recommend visualizations or subsequent interactions. Future work may explore ways to complement user-driven exploration with more prescriptive guidance besides expanding on existing research on managing notebook provenance [7].

Towards scalable data science. The interaction-aware MAGNETON widgets can serve as the presentation layer of provenance-preserving end-to-end systems for data science [33, 34]. However,

for large-scale systems, the latency of rendering widgets remains a bottleneck. Recent work on enhancing the scalability of Vega visualization generation introduced automatic server-side scaling via partitioning strategies [25]. Similar strategies can be adopted by MAGNETON. Other approaches that may be employed to redesign widgets for scale include applying classical database optimization techniques such as caching, pre-fetching, indexing, materialization, and incremental view maintenance on the server side.

The role of collaboration in widget design. Since notebooks can be collaborative, MAGNETON widgets may need to accommodate shared workflows. Collaboration introduces new challenges, such as enforcing access control mechanisms, characterizing the role of users, and instrumenting conflict resolution techniques [35], all of which indicate the possibility of interesting future research. Within the collaborative setting, another dimension is the plasticity of interfaces [28] — the perceived value of an interface may vary across stakeholders. To this end, another extension could be to equip MAGNETON widgets with cross-platform capabilities [5, 34].

6 CONCLUSION

We contribute MAGNETON, a framework for composing interaction history-aware and customizable widgets to enable transparent, reusable, and expressive data science workflows in computational notebooks. Case studies conducted in the industry setting demonstrated how MAGNETON widgets enabled efficient graph analysis workflows compared to participants' previous experience by surfacing inconsistencies and data quality issues. The built-in interaction history enabled sharing of insights and data among various steps within a project. The shared action features empowered users to customize the widget, thereby reducing dependency on developers. However, similar to any new framework, MAGNETON exhibited several limitations related to incomplete documentation and gaps in widget feature design, among others. These challenges encourage interesting future research.

ACKNOWLEDGMENTS

We would like to thank Rafael Li Chen for his assistance in setting up the initial project infrastructure. Our study participants set aside time for interviews and reflection. We're grateful for their contributions to this work.

REFERENCES

- [1] Eytan Adar. 2006. GUESS: a language and interface for graph exploration. In *Proceedings of the SIGCHI conference on Human Factors in computing systems*. ACM, New York, NY, USA, 791–800.
- [2] Sara Alspaugh, Nava Zokaei, Andrea Liu, Cindy Jin, and Marti A. Hearst. 2019. Futzing and Moseying: Interviews with Professional Data Analysts on Exploration Practices. *IEEE Transactions on Visualization and Computer Graphics* 25, 1 (2019), 22–31. <https://doi.org/10.1109/TVCG.2018.2865040>
- [3] Robert Amar, James Eagan, and John Stasko. 2005. Low-level components of analytic activity in information visualization. In *IEEE Symposium on Information Visualization, 2005. INFOVIS 2005*. IEEE, New York, NY, USA, 111–117.
- [4] Andrea Batch and Niklas Elmqvist. 2017. The interactive visualization gap in initial exploratory data analysis. *IEEE transactions on visualization and computer graphics* 24, 1 (2017), 278–287.
- [5] Alex Bäuerle, Ángel Alexander Cabrera, Fred Hohman, Megan Maher, David Koski, Xavier Suau, Titus Barik, and Dominik Moritz. 2022. Symphony: Composing Interactive Interfaces for Machine Learning. In *CHI Conference on Human Factors in Computing Systems*. ACM, New York, NY, USA, 1–14.
- [6] Mangesh Bendre, Tana Wattanawaroon, Sajjadur Rahman, Kelly Mack, Yuyang Liu, Shichu Zhu, Yu Lu, Ping-Jing Yang, Xinyan Zhou, Kevin Chen-Chuan Chang, et al. 2019. Faster, higher, stronger: Redesigning spreadsheets for scale. In *2019 IEEE 35th International Conference on Data Engineering (ICDE)*. IEEE, New York, NY, USA, 1972–1975.
- [7] Michael Brachmann and William Spoth. 2020. Your notebook is not crumbly enough, REPLace it. In *Conference on Innovative Data Systems Research (CIDR)*. CIDRDB, Chaminade, California, 1–8.
- [8] Souti Chattopadhyay, Ishita Prasad, Austin Z Henley, Anita Sarma, and Titus Barik. 2020. What's wrong with computational notebooks? Pain points, needs, and design opportunities. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*. ACM, New York, NY, USA, 1–12.
- [9] Will Epperson, Doris Jung-Lin Lee, Leijie Wang, Kunal Agarwal, Aditya G Parameswaran, Dominik Moritz, and Adam Perer. 2022. Leveraging Analysis History for Improved In Situ Visualization Recommendation. *Computer Graphics Forum* 41, 3 (2022), 145–155.
- [10] Max Franz, Christian T Lopes, Gerardo Huck, Yue Dong, Onur Sumer, and Gary D Bader. 2016. Cytoscape.js: a graph theory library for visualisation and analysis. *Bioinformatics* 32, 2 (2016), 309–311.
- [11] Graphileon. 2022. *Graphileon*. Graphileon. Retrieved January 19, 2023 from <https://graphileon.com/>
- [12] Peter Griggs, Cagatay Demiralp, and Sajjadur Rahman. 2021. Towards integrated, interactive, and extensible text data analytics with Leam. In *Proceedings of the Second Workshop on Data Science with Human in the Loop: Language Advances*. Association for Computational Linguistics, Online, 52–58. <https://doi.org/10.18653/v1/2021.dash-1.9>
- [13] Holoviz. 2022. *Panel*. Panel. Retrieved January 19, 2023 from <https://panel.holoviz.org/>
- [14] IDOM. 2022. *IDOM - a declarative Python package for building highly interactive user interfaces*. IDOM. Retrieved January 19, 2023 from <https://ryanmorshead.com/articles/2021/idom-react-but-its-python/article/>
- [15] Observable Inc. 2022. *Observable*. Observable. Retrieved January 19, 2023 from <https://observablehq.com/>
- [16] Streamlit Inc. 2022. *Streamlit*. Streamlit. Retrieved January 19, 2023 from <https://streamlit.io/>
- [17] Jupyter. 2022. *IPyWidgets*. Jupyter. Retrieved January 19, 2023 from <https://ipywidgets.readthedocs.io/en/stable/>
- [18] Mary Beth Kery, Amber Horvath, and Brad Myers. 2017. Variolite: Supporting Exploratory Programming by Data Scientists. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*. ACM, New York, NY, USA, 1265–1276.
- [19] Mary Beth Kery, Bonnie E John, Patrick O'Flaherty, Amber Horvath, and Brad A Myers. 2019. Towards effective foraging by data scientists to find past analysis choices. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. ACM, New York, NY, USA, 1–13.
- [20] Mary Beth Kery and Brad A. Myers. 2018. Interactions for Untangling Messy History in a Computational Notebook. In *2018 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. IEEE, New York, NY, USA, 147–155.
- [21] Mary Beth Kery, Marissa Radensky, Mahima Arya, Bonnie E John, and Brad A Myers. 2018. The story in the notebook: Exploratory data science using a literate programming tool. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*. ACM, New York, NY, USA, 1–11.
- [22] Mary Beth Kery, Donghao Ren, Fred Hohman, Dominik Moritz, Kanit Wongsuphasawat, and Kayur Patel. 2020. mage: Fluid moves between code and graphical work in computational notebooks. In *Proceedings of the 33rd Annual ACM Symposium on User Interface Software and Technology*. ACM, New York, NY, USA, 140–151.
- [23] Thomas Kluyver, Benjamin Ragan-Kelley, Fernando Pérez, Brian E Granger, Matthias Bussonnier, Jonathan Frederic, Kyle Kelley, Jessica B Hamrick, Jason Grout, Sylvain Corlay, et al. 2016. *Jupyter Notebooks—a publishing format for reproducible computational workflows*. Vol. 2016. IOS Press, Amsterdam, Netherlands.
- [24] Larry G Kontosh. 1999. An occupational information system for the 21st century: The development of the O* NET. *Journal of Applied Rehabilitation Counseling* 30, 2 (1999), 43.
- [25] Nicolas Kruchten, Jon Mease, and Dominik Moritz. 2022. VegaFusion: Automatic Server-Side Scaling for Interactive Vega Visualizations. In *2022 IEEE Visualization and Visual Analytics (VIS)*. IEEE, New York, NY, USA, 11–15.
- [26] Zhicheng Liu, Biye Jiang, and Jeffrey Heer. 2013. imMens: Real-time visual querying of big data. In *Computer Graphics Forum*. Wiley Online Library, New York, NY, USA, 421–430.
- [27] Justin J Miller. 2013. Graph database applications and concepts with Neo4j. In *Proceedings of the southern association for information systems conference, Atlanta, GA, USA*. AIS eLibrary, WWW, 1–7.
- [28] Samir Passi and Steven J Jackson. 2018. Trust in data science: Collaboration, translation, and accountability in corporate data science projects. *Proceedings of the ACM on Human-Computer Interaction* 2, CSCW (2018), 1–28.
- [29] Plotly. 2022. *Dash*. Plotly. Retrieved January 19, 2023 from <https://plotly.com/dash/>
- [30] Plotly. 2022. *Low-code Data Apps*. Plotly. Retrieved January 19, 2023 from <https://plotly.com/>

- [31] Sajjadur Rahman, Mangesh Bendre, Yuyang Liu, Shichu Zhu, Zhaoyuan Su, Karrie Karahalios, and Aditya Parameswaran. 2021. NOAH: Interactive Spreadsheet Exploration with Dynamic Hierarchical Overviews. *Proceedings of the VLDB Endowment* 14, 6 (2021), 970–983.
- [32] Sajjadur Rahman, Peter Griggs, and Çağatay Demiralp. 2021. Leam: An Interactive System for In-situ Visual Text Analysis. In *CIDR*. CIDRDB, cidrdb.org, 1–7.
- [33] Sajjadur Rahman and Eser Kandogan. 2022. Characterizing Practices, Limitations, and Opportunities Related to Text Information Extraction Workflows: A Human-in-the-Loop Perspective. In *CHI Conference on Human Factors in Computing Systems* (New Orleans, LA, USA) (*CHI '22*). Association for Computing Machinery, New York, NY, USA, Article 628, 15 pages. <https://doi.org/10.1145/3491102.3502068>
- [34] Sajjadur Rahman, Hannah Kim, Dan Zhang, Estevam Hruschka, and Eser Kandogan. 2022. Towards Multifaceted Human-Centered AI. *ArXiv abs/2301.03656* (2022), 1–2.
- [35] Sajjadur Rahman, Pao Siangliulue, and Adam Marcus. 2020. MixTAPE: Mixed-initiative Team Action Plan Creation Through Semi-structured Notes, Automatic Task Generation, and Task Classification. *Proceedings of the ACM on Human-Computer Interaction* 4, CSCW2 (2020), 1–26.
- [36] ReactJS. 2022. *React Framework*. ReactJS. Retrieved January 19, 2023 from <https://reactjs.org/>
- [37] Arvind Satyanarayan and Jeffrey Heer. 2014. Lyra: An interactive visualization design environment. In *Computer Graphics Forum*. Wiley Online Library, New York, NY, USA, 351–360.
- [38] Arvind Satyanarayan, Dominik Moritz, Kanit Wongsuphasawat, and Jeffrey Heer. 2016. Vega-lite: A grammar of interactive graphics. *IEEE transactions on visualization and computer graphics* 23, 1 (2016), 341–350.
- [39] TypeScript. 2022. *TypeScript Language*. TypeScript. Retrieved January 19, 2023 from <https://www.typescriptlang.org/>
- [40] Jacob VanderPlas, Brian Granger, Jeffrey Heer, Dominik Moritz, Kanit Wongsuphasawat, Arvind Satyanarayan, Eitan Lees, Ilia Timofeev, Ben Welsh, and Scott Sievert. 2018. Altair: interactive statistical visualizations for Python. *Journal of open source software* 3, 32 (2018), 1057.
- [41] Kanit Wongsuphasawat, Yang Liu, and Jeffrey Heer. 2019. Goals, Process, and Challenges of Exploratory Data Analysis: An Interview Study. *ArXiv abs/1911.00568* (2019), 1–10.
- [42] Yifan Wu, Joseph M Hellerstein, and Arvind Satyanarayan. 2020. B2: Bridging code and interactive visualization in computational notebooks. In *Proceedings of the 33rd Annual ACM Symposium on User Interface Software and Technology*. Association for Computing Machinery, New York, NY, USA, 152–165.
- [43] Ji Soo Yi, Youn ah Kang, John Stasko, and Julie A Jacko. 2007. Toward a deeper understanding of the role of interaction in information visualization. *IEEE transactions on visualization and computer graphics* 13, 6 (2007), 1224–1231.
- [44] Amy X Zhang, Michael Muller, and Dakuo Wang. 2020. How do data science workers collaborate? roles, workflows, and tools. *Proceedings of the ACM on Human-Computer Interaction* 4, CSCW1 (2020), 1–23.
- [45] Dan Zhang, Hannah Kim, Rafael Li Chen, Eser Kandogan, and Estevam Hruschka. 2022. MEGAnno: Exploratory Labeling for NLP in Computational Notebooks. In *Proceedings of the Fourth Workshop on Data Science with Human-in-the-Loop (Language Advances)*. Association for Computational Linguistics, Abu Dhabi, United Arab Emirates (Hybrid), 1–7. <https://aclanthology.org/2022.dash-1.1>
- [46] Xiong Zhang, Jonathan Engel, Sara Evensen, Yuliang Li, Çağatay Demiralp, and Wang-Chiew Tan. 2020. Teddy: A System for Interactive Review Analysis. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) (*CHI '20*). Association for Computing Machinery, New York, NY, USA, 1–13. <https://doi.org/10.1145/3313831.3376235>